

Programming for Essential Digital Skills, Chapter 6

Christoph Walsh

2026

Defining Functions

If we wanted to create our own function that adds one to its input, i.e.:

$$f(x) = x + 1$$

We can do:

```
def addone(x):  
    y = x + 1  
    return y  
addone(2)
```

3

Again, indentation matters!

Local vs Global Variables

- The `y` that we create inside the function is not available outside it.
- If we were to run the code below, we would get an error saying `y` is not defined.
- The `y` is only *local* to the function, and is not available globally.

```
x = 2
def addone(x):
    y = x + 1
    return y
addone(2)
y
```

Local vs Global Variables

- If we wanted to access `y` outside the function, we could do the following.
- Generally this is not recommended practice, but it can be useful for debugging.

```
x = 2
def addone(x):
    global y
    y = x + 1
    return y
addone(2)
```

3

y

3

If-Else Statements

- Suppose we wanted to create our own `abs()` function from scratch:

$$|x| = \begin{cases} -x & \text{if } x < 0 \\ x & \text{otherwise} \end{cases}$$

- We can use an if-else statement to do this:

```
def my_abs(x):  
    if x < 0:  
        return -x  
    else:  
        return x  
[my_abs(i) for i in [-2, 0, 3]]
```

```
[2, 0, 3]
```

If-Else If-Else Statements

- Now let's make the $sgn(x)$ function which gives the sign in front of a number:

$$sgn(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ +1 & \text{otherwise} \end{cases}$$

```
def sign(x):  
    if x < 0:  
        return -1  
    else:  
        if x == 0:  
            return 0  
        else:  
            return 1  
[sign(i) for i in [-2, 0, 3]]
```

```
[-1, 0, 1]
```

If-Else If-Else Statements

- Instead of nesting multiple if-else statements, we can use `elif`:

```
def sign(x):  
    if x < 0:  
        return -1  
    elif x == 0:  
        return 0  
    else:  
        return 1  
[sign(i) for i in [-2, 0, 3]]
```

```
[-1, 0, 1]
```

While Loops

- When you want to repeat commands for an indeterminate amount of time we can use a while loop.
- A while loop repeats a command as long as a condition is True:

```
counter = 0
while counter < 3:
    print('hello')
    counter += 1
```

hello
hello
hello

- Using while True: repeats the commands forever.

Using While Loops to Approximate

- The Ancient Greeks used the following method to find the square root of a number.
- They began with an initial guess $y_0 = \frac{x}{2}$ for $\sqrt{x}$.
- They then updated the guess repeatedly where the n th guess is:

$$y_n = \frac{1}{2} \left(y_{n-1} + \frac{x}{y_{n-1}} \right)$$

- For $x = 2$ (where $\sqrt{2} = 1.414214$), this approach yields:

$$(y_0, y_1, y_2, y_3) = (1, 1.5, 1.416667, 1.414216)$$

- It converges very quickly to very close to the solution!
- We can use a while loop to approximate square roots, where we let the loop run until the answer stops changing very much.

Using While Loops to Approximate: The Square Root

```
def my_sqrt(x, tol=0.000001):  
    y = x / 2 # Initial guess  
    dist = tol + 1 # Initialize distance  
    while dist > tol: # Repeat until dist <= tol  
        y_old = y # Previous guess  
        y = (y_old + x / y_old) / 2 # Update guess  
        dist = abs(y - y_old) # Distance from previous guess  
    return y  
my_sqrt(2, 0.1) # Loose tolerance
```

1.4166666666666665

```
my_sqrt(2) # Default (tighter) tolerance
```

1.414213562373095